MEAM 620 Project 1 Phase 4 Report

Team 5: Sanghyub(Paul) Lee, Angelina Zhang, Parra Wang, Tianyi Xia

I. Introduction and System Overview

Our project goal was to deploy the planning, trajectory generation, and feedback control algorithms developed in Project 1 on a real Crazyflie quadrotor and evaluate their performance in a motion-capture laboratory environment. Across the two in-person labs, we demonstrated two main capabilities. In Lab 1, we implemented and tuned our tracking controller and waypoint trajectory generator on hardware, validating stable hover and waypoint-following performance. In Lab 2, we extended the system to autonomous maze flight by integrating path planning, trajectory generation, and feedback control to safely navigate through a known cluttered environment.

The experimental hardware platform consisted of a Crazyflie 2.0 quadrotor, a VICON motion-capture arena, and a laboratory desktop computer. In this setup, sensing was handled externally by the VICON system, which provided real-time state measurements to the lab computer. High-level computation was executed on the desktop computer, with path planning handled by `graph_search.py`, path post-processing and trajectory generation by `world_traj.py`, and the SE(3) trajectory tracking by `se3_control.py`. The computed motor commands, specifically the desired thrust u_1 and control moments M derived from the tracking controller, were then sent to the Crazyflie via a radio link. The quadrotor executed these commands through its onboard actuation. Additionally, during each flight, external VICON measurements, issued commands, and system states were continuously routed to the lab computer and recorded into `rosbag` files for post-processing and performance analysis.

II. Controller

The controller used in this project is a geometric SE(3) controller. The outer loop is a PD linear controller regulating translation in Cartesian space, while the inner loop regulates attitude using a nonlinear PD controller so that the thrust vector aligns with the desired direction. In the CrazyFlie experiments, yaw tracking was disabled, so the controller mainly regulates position in x, y, z without enforcing heading. In simulation, yaw can be generated from the trajectory, but it is not used since it introduces additional coupling without improving tracking performance.

A. Position Controller

The position and velocity errors are defined as:

$$e_x = x - x_T, \quad e_v = v - v_T \tag{1}$$

The commanded acceleration is:

$$a_{cmd} = a_T - K_d e_v - K_p e_x \tag{2}$$

where a_T is the feedforward acceleration. The desired force is:

$$F_{des} = m(a_{cmd} + g e_3) \tag{3}$$

The K_p term pulls the system back toward the desired trajectory, while K_d provides damping and reduces overshoot.

For the CrazyFlie, the gains are:

$$K_p = diag(4.5, 4.5, 11.0), \quad K_d = diag(3.0, 3.0, 5.5) \tag{4}$$

and in simulation:

$$K_p = diag(12.0, 12.0, 6.5), \quad K_d = diag(8.0, 8.0, 6.0) \tag{5}$$

The larger gain in the z-direction reflects the need to counter gravity, while simulation allows larger gains due to ideal assumptions.

B. Attitude and Thrust Controller

From the desired force, the desired thrust direction is:

$$b_{3,des} = \frac{F_{des}}{\|F_{des}\|} \tag{6}$$

Using the heading vector

$$a_\psi = [\cos(yaw), \sin(yaw), 0] \tag{7}$$

the desired body frame is constructed as:

$$b_{2,des} = b_{3,des} \times a_\psi, \quad b_{1,des} = b_{2,des} \times b_{3,des} \tag{8}$$

$$R_{des} = [b_{1,des}, b_{2,des}, b_{3,des}] \tag{9}$$

The attitude and angular velocity errors are:

$$e_R = \frac{1}{2}(R_{des}^T R - R^T R_{des}) \quad e_\omega = \omega - \omega_{des} \tag{10}$$

The control moment and thrust are computed as:

$$M = I(-K_R e_R - K_\omega e_\omega), \quad u_1 = F_{des} \cdot (R e_3) \tag{11}$$

These are mapped to motor commands through a control allocation matrix.

C. Discussion: Simulation vs Experiment

The main difference between simulation and real experiments comes from how idealized the model is. In simulation, the system assumes perfect state estimation, instantaneous actuator response, and no disturbances. This allows the use of larger gains, especially in the rotational loop, to aggressively align the thrust vector with the desired force.

In contrast, the real CrazyFlie must account for actuator dynamics, sensor noise, and modeling uncertainties. The motors cannot respond instantaneously, so high gains can cause the controller to react faster than the system can respond, leading to overshoot or oscillations. Additionally, sensor noise and estimation errors are amplified by high gains, which can result in jittery behavior.

The real system also operates under uncertainties such as aerodynamic disturbances and battery voltage variation. These factors reduce robustness if the controller is too aggressive. Therefore, more conservative and balanced gains are required in practice. Yaw control is also disabled in experiments since it is not essential for trajectory tracking and can introduce additional coupling that extends the time to stabilize and turning performances without more efforts for tuning.

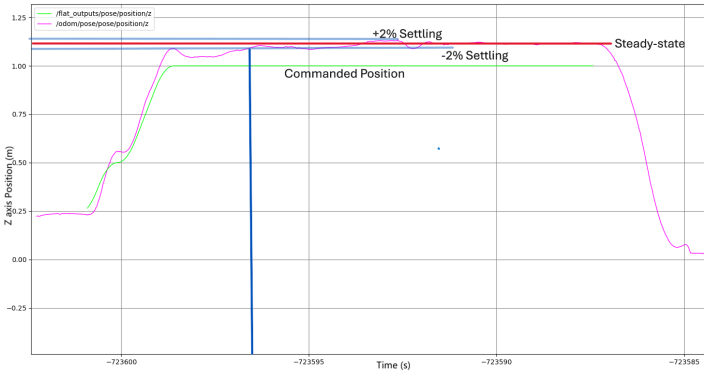


Fig. 1: Z-axis position response over time. The green curve represents the commanded position, while the purple curve represents the actual measured position. The $\pm 2\%$ bounds are defined around the steady-state value of the actual response and are used to determine the settling time.

D. Controller Performance

Fig. 1 shows the z-axis position response for one trajectory segment.

The system tracks a step input from approximately $0.25\,m$ to $1.0\,m$. The actual response rises quickly and reaches the vicinity of the desired value within about $1 \sim 2\,s$, indicating a fast rise time. A small overshoot is observed, which suggests the system is slightly underdamped but still well controlled.

The steady-state position is approximately $1.10 \sim 1.12\,m$, which is higher than the commanded value of $1.0\,m$. This indicates a steady-state offset in the system. This offset is likely caused by modeling inaccuracies, thrust bias, estimation errors in the real system, and most significant, the lack of the integration term in the controller.

The $\pm 2\%$ settling bounds are defined around the actual steady-state value rather than the commanded position. The response enters this region after approximately $2 \sim 3\,s$ and remains within it, which defines the settling time.

After settling, small oscillations are observed around the steady-state value. These oscillations are relatively low in amplitude and are likely caused by sensor noise and actuator dynamics rather than instability in the controller. During the descent phase, the system responds smoothly without significant oscillations, indicating consistent damping behavior even during rapid changes in reference. Overall, the system exhibits fast rise time, moderate settling time, and stable tracking performance. The presence of steady-state offset suggests that the controller is primarily a PD controller using gains K_p and K_d, without integral action to eliminate steady-state error.

E. Trajectory Generation

Given a known map, the waypoint planner first computed a collision-free dense path using A* search on a 3D occupancy grid. The environment was discretized at a resolution of $\Delta = [0.25,\ 0.25,\ 0.25]$ m and the occupied cells were inflated by a safety margin of $m = 0.15$ m. Each free voxel was treated as a graph node, and the search was performed over a 26-connected neighborhood. The edge cost between neighboring voxels was taken as the Euclidean metric distance

$$d = \|\mathbf{p}_i - \mathbf{p}_j\| = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2} \tag{12}$$

The A* heuristic was the straight-line distance from the current voxel center to the goal voxel center,

$$h(i) = \|p_i - p_g\| = \sqrt{(x - x_g)^2 + (y - y_g)^2 + (z - z_g)^2} \tag{13}$$

After backtracking through the parent pointers, the resulting dense path was converted from voxel centers into a sequence of metric waypoints, with the first and last points replaced by the exact start and goal positions.

To improve the quality of the path before trajectory generation, the dense A* output was post-processed into a smaller set of sparse waypoints. A greedy shortcutting method was used: from the current point, the algorithm attempted to connect directly to the farthest reachable future point, subject to collision checking of the straight segment. In the implementation, the shortcut search used a lookahead of 4 path points. Each candidate straight segment was checked by sampling points along the line with spacing step $= 0.5 \min(\Delta) = 0.125$ m. Additional cleanup rules were then applied. Consecutive waypoints separated by less than 0.15 m were removed, very short final segments shorter than 0.35 m were eliminated, and points too close to the goal were pruned using `goal_keep_radius` $= 0.6$ m. This post-processing step significantly reduced unnecessary voxel-scale turns and produced a much cleaner waypoint set for spline generation.

Time was allocated between successive waypoints using a distance-based rule. For waypoint sequence $\{P_0, P_1, \ldots, P_s\}$, the duration of segment k was chosen as

$$T_k = \max\left(T_{\min}, \frac{\|P_{k+1} - P_k\|}{v_{\text{nom}}}\right) \tag{14}$$

with nominal speed $v_{\text{nom}} = 0.5$ m/s and minimum segment duration $T_{\min} = 0.1$ s. This timing method assigns more time to longer segments while preventing unrealistically short time allocations on very small segments.

The trajectory generator uses a piecewise quintic polynomial minimum-jerk spline. Each segment is represented by a 5th-order polynomial, with boundary conditions on position, zero initial/final velocity and acceleration, and continuity of velocity, acceleration, jerk, and snap at internal waypoints. For each coordinate,

$$x^*(t) = \arg\min_{x(t)} \int_0^T (\dddot{x}(t))^2\, dt. \tag{15}$$

With $\mathcal{L} = (\dddot{x})^2$, the Euler–Lagrange equation reduces to $x^{(6)}(t) = 0$. Therefore, the solution in each segment must be a quintic polynomial,

$$x(t) = c_5 t^5 + c_4 t^4 + c_3 t^3 + c_2 t^2 + c_1 t + c_0 \tag{16}$$

and similarly for $y(t)$ and $z(t)$. The coefficients were found by solving one linear system per axis using the following constraints. First, each segment was constrained to pass through its endpoints:

$$r_k(0) = P_k, \qquad r_k(T_k) = P_{k+1}.$$

Second, zero initial and final velocity and acceleration were imposed:

$$\dot{r}_0(0) = 0, \qquad \ddot{r}_0(0) = 0,$$

$$\dot{r}_{s-1}(T_{s-1}) = 0, \qquad \ddot{r}_{s-1}(T_{s-1}) = 0.$$

Finally, at every internal waypoint, continuity was enforced up to the fourth derivative where $n = 4$:

$$r_k^{(n)}(T_k) = r_{n+1}^{(4)}(0).$$

Thus, the trajectory was continuous in position, velocity, acceleration, jerk, and snap across segment boundaries. Once the coefficients were solved, the trajectory update function returned the outputs required by the controller x, $\dot{x}$, $\ddot{x}$, $\dddot{x}$, $\ddddot{x}$, along with zero yaw and yaw rate.

F. Smoothness and Feasibility

This trajectory is a major improvement over the original lab 1 constant-speed linear waypoint trajectory, which has discontinuous velocity at corners and thus effectively commands infinite acceleration at waypoints. The quintic spline produced continuous derivatives.

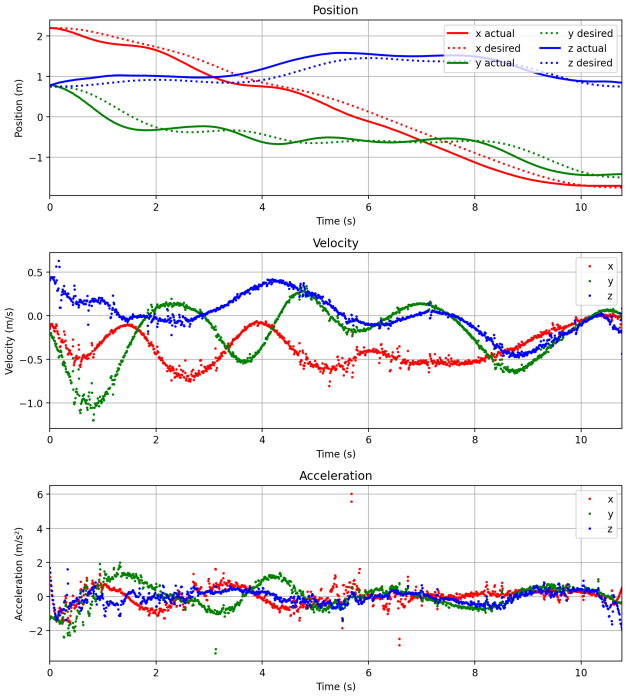


Fig. 2: Experimental tracking results for Maze 3.

In Fig. 2, the desired position curves vary continuously in x, y, and z, which is consistent with a spline-based minimum-jerk style trajectory. The actual position generally follows the desired position, showing that the quadrotor was able to track the planned path. The velocity profiles are also continuous, without abrupt jumps, further indicating a smooth reference trajectory. The acceleration remains moderate for most of the run, although there are a few brief spike, indicating moments where the controller had to work harder to recover tracking error or respond to disturbances. But as these spikes are brief rather than sustained, the trajectory still appears practically feasible for the platform.

III. Maze Flight Experiments

Before analyzing and reporting the maze flight experiments, we must describe the retuning process conducted before running them. The initial K_p and K_d that were tuned during lab 1 using stepz.json, stepy.json, and evaluation.json had to be retuned at the start of lab 2.

After lab 1:

$$K_p = diag[4.0, 4.0, 6.5], \quad K_d = diag[2.0, 2.0, 6.0]$$

After lab 2:

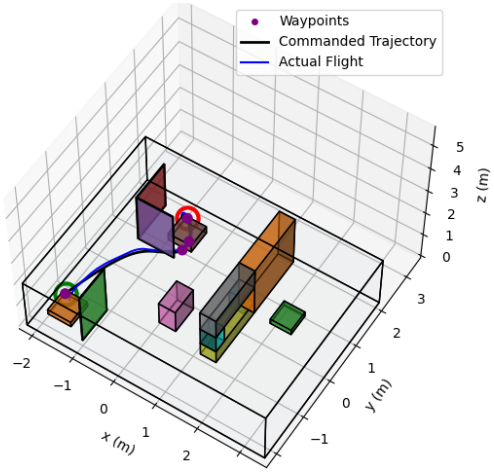
$$K_p = diag[4.5, 4.5, 11.0], \quad K_d = diag[3.0, 3.0, 5.5]$$

We have used the same code, but our previously successful tuned parameters did not perform well. We noticed an offset in altitude (z) positions, so we increased the gain and tuned the lateral (x,y) positions as well. The derivative gains in x and y were increased to damp oscillation, and the derivative gains in z were slightly adjusted to balance the much higher altitude proportional gain. TA in our sessions attributed the change in the CrazyFlie's motion capture marker layout to the necessary retuning of our gains. The marker placement has been updated for better tracking in the VICON system. During lab 2, the tuned resolution is 0.25m in all directions, and the tuned margin is 0.15 meters. As shown in Fig. 3 and in the **video**, we successfully tuned our parameters to achieve safe, less computationally intensive waypoint generation and planned trajectories to produce both simulated and real-world flight paths that successfully carried out the flight in all three maps.

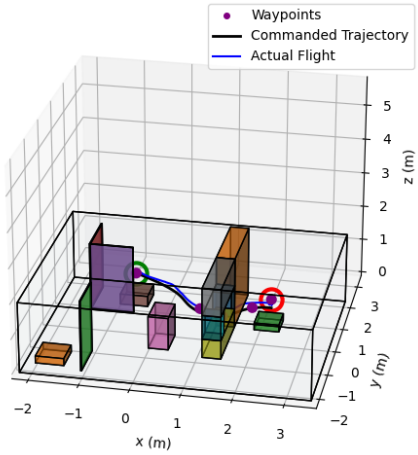
TABLE I: Position Tracking RMSE (Commanded vs. Actual)

Maze	RMSE$_x$ (m)	RMSE$_y$ (m)	RMSE$_z$ (m)	RMSE$_{3D}$ (m)
Maze 1	0.0547	0.0503	0.1162	0.1380
Maze 2	0.0258	0.0356	0.1126	0.1208
Maze 3	0.0402	0.0574	0.1203	0.1392

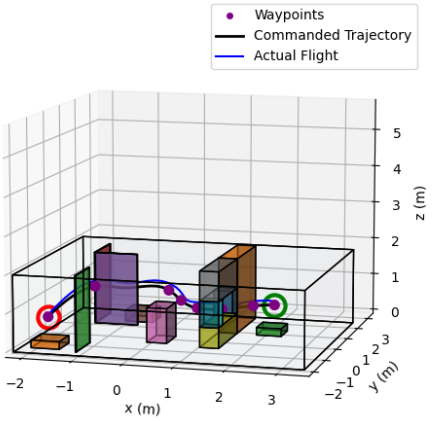
From Tab. I and Fig. 4 we can see that our controller achieves generally high, consistent position accuracy across all three maps. The lateral accuracies exhibit surprising values, with RMSEs ranging from 0.02 to 0.05 m. The altitudal accuracies exhibit consistent RMSE with about 0.12 m of RMSE across all maps. The low standard deviation of altitudial error (0.0394 m (average of all three trials)) in the Z-axis shows that this offset error is consistent and repetitive rather than oscillatory. The constant Z positional offset across all three independent trials is most likely due to a mismatch between the VICON world frame Z value and the drone's onboard reference frame, resulting in a constant vertical offset across all three Maze flights. Another probable cause is that we have not implemented a PID controller to reduce these steady-state errors. The PID controller would most likely eliminate the constant offset that exists in our flights. With these performance metrics, we can evaluate how aggressive our parameters were for the three flights. With the total 3D RMSE around 0.13m, our selected margin value (0.15m) was aggressively chosen. With the constant offset in mind, we need to increase our margins and resolutions to be much more conservative. Therefore, if we had one more session in the lab, we would develop a PID controller to evaluate its performance against our existing controller and see if we can eliminate the consistent altitudinal position errors. Also, we want to increase the speed and test the stability of our controller more rigorously. To test at high velocities, we believe we need to introduce yaw control and use minimum-snap trajectories to generate smoother flights and reduce oscillations. Therefore, we would want to implement several different strategies and evaluate each performance if we had another lab.



(a) Maze 1 Trajectory

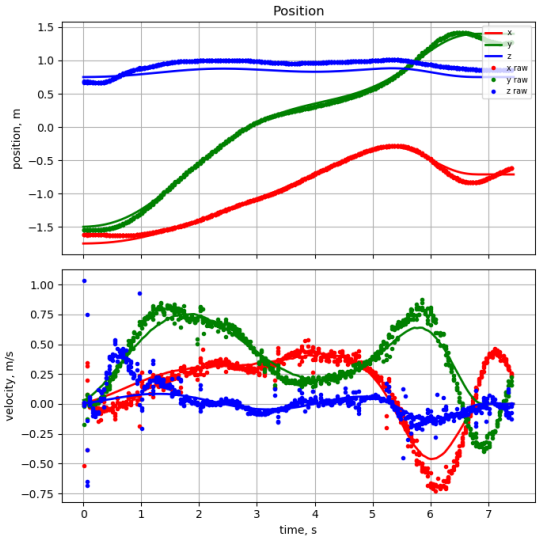


(b) Maze 2 Trajectory

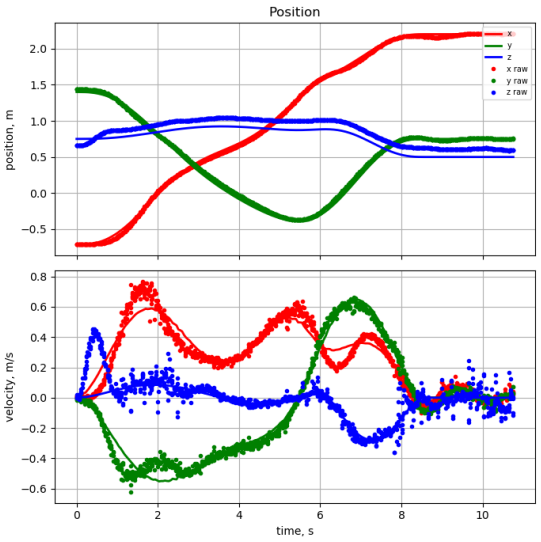


(c) Maze 3 Trajectory

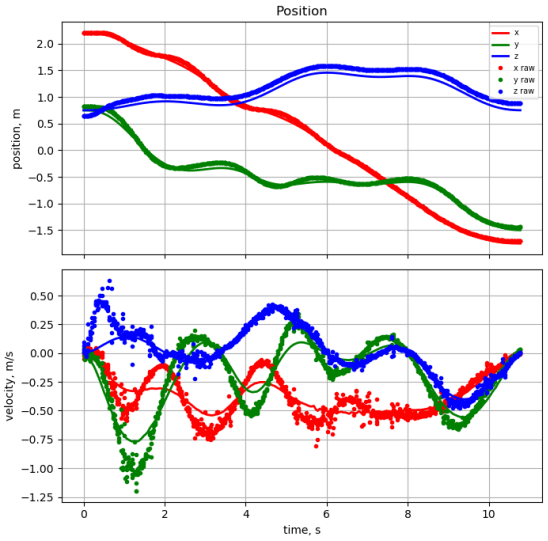
Fig. 3: 3D flight trajectories for the three maze environments, illustrating waypoint generation (red dots), planned trajectories (black line), and actual recorded paths (blue line).



(a) Maze 1 Performance



(b) Maze 2 Performance



(c) Maze 3 Performance

Fig. 4: Position and velocity performance for the three mazes, comparing commanded values (line) and actual flight data (dots).